

苏州大学实验报告

院、系	未来科学与工程学院	年级专业	未来 22 人工智能	姓名	张羽鑫	学号	2262404049
课程名称	综合智能项目实践					成绩	
指导教师	张得天	同组实验者	邹杰 苏靖轩 袁镜洛	实验日期	2025-3-14		

报 告 名 称 《项目设计文档》

1. 系统架构设计

1.1 技术栈

🌈 前端：PyQt

1. 用户交互界面

- 构建跨平台桌面应用程序, 提供图像上传/摄像头调用、分类结果显示、历史记录查询等可视化操作界面
- 实现动态数据展示: 实时渲染 YOLO 检测框、置信度、垃圾分类结果及对应处理建议
- 集成多线程机制处理图像上传与模型推理的异步操作, 避免界面卡顿

2. 本地设备对接

- 调用系统摄像头进行实时视频流采集 (使用 OpenCV-Python 集成)
- 管理本地图片/视频文件选择器, 支持常见图像格式 (JPG/PNG/MP4) 的读取与预处理
- 使用多种输入源: 从图片、视频、摄像头、文件夹和网络摄像头等多种输入源进行目标检测, 满足不同场景下的使用需求

3. 通信模块

- 通过 HTTP 协议与 Flask 后端交互, 使用 requests 库发图片数据
- 处理 API 响应数据并格式化显示, 异常时提供友好错误提示 (如网络中断、模型超时等)

🚦 后端：Flask

1. RESTful API 服务

- 设计标准化接口：/detect（检测接口）、/history（历史记录）、/class-info（分类标准查询）
- 使用 flask-restx 扩展实现 Swagger 文档自动生成，便于前后端协作调试

2. 业务逻辑处理

- 接收 Base64 编码或二进制图像数据，进行尺寸校验（限制最大 10MB）和格式验证
- 调用 PyTorch 推理服务时加入熔断机制，当模型响应超时（>5s）自动返回服务不可用状态

3. 数据中转枢纽

- 使用 SQLAlchemy ORM 对接 MySQL，通过连接池管理数据库访问
- 缓存高频查询数据：将垃圾分类标准（如可回收/有害/厨余/其他）存入 Redis，降低数据库存储与检索压力

🚦 智能模块：PyTorch

1. YOLO 模型部署

- 使用 YOLO11-MSCA 版本，在保持 85%+mAP 精度的同时将模型尺寸压缩至 40MB 以内
- 实现多尺度推理：自动识别输入分辨率，动态调整 grid size（支持 640p/1080p 输入）

2. 垃圾分类优化

- 自定义数据增强：针对反光垃圾袋、堆叠物品等场景增加过曝模拟、局部遮挡增强、马赛克增强、随机剪裁增强、透视变化增强等
- 修改 NMS 后处理模块，设置动态置信度阈值（白天 0.5/夜间 0.4）以适应不同光照条件

3. 性能加速

- 启用 TensorRT 加速，在 NVIDIA GPU 环境实现 30FPS+ 的推理速度
- 使用 ONNX Runtime 进行 CPU 推理优化，在 Intel i5 处理器上达到 25FPS 实时性

🚦 MySQL（数据库）

1. 数据存储设计

- 核心表结构：
 - 用户表(User): user_id、username、password_hash、email、create_time、last_login
 - 垃圾分类结果表(DetectionResult): result_id、user_id、image_path、detected_class、confidence、detect_time、model_version
 - 分类类别表(GarbageCategory): category_id、class_name、display_name、parent_id、disposal_guide
 - 模型版本表(ModelVersion): version_id、version_name、model_path、

update_time、is_active

- 用户反馈表 (UserFeedback): feedback_id、result_id、feedback_type、content、submit_time

- 建立联合索引: 对多个表的 (如用户 ID+分类结果) 建立组合索引

2. 查询优化

- 历史记录分页查询使用游标分页技术, 避免 LIMIT OFFSET 的性能瓶颈
- 高频访问的垃圾分类标准数据设置内存表 (MEMORY Storage Engine)

3. 数据安全

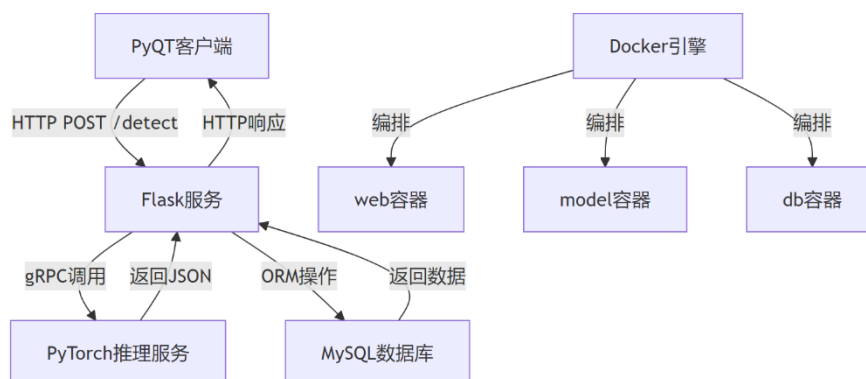
- 启用 SSL 加密连接, 配置数据库审计日志
- 定期执行逻辑备份 (mysqldump), 保留近期备份

🌈 Docker (部署)

1. 容器化方案

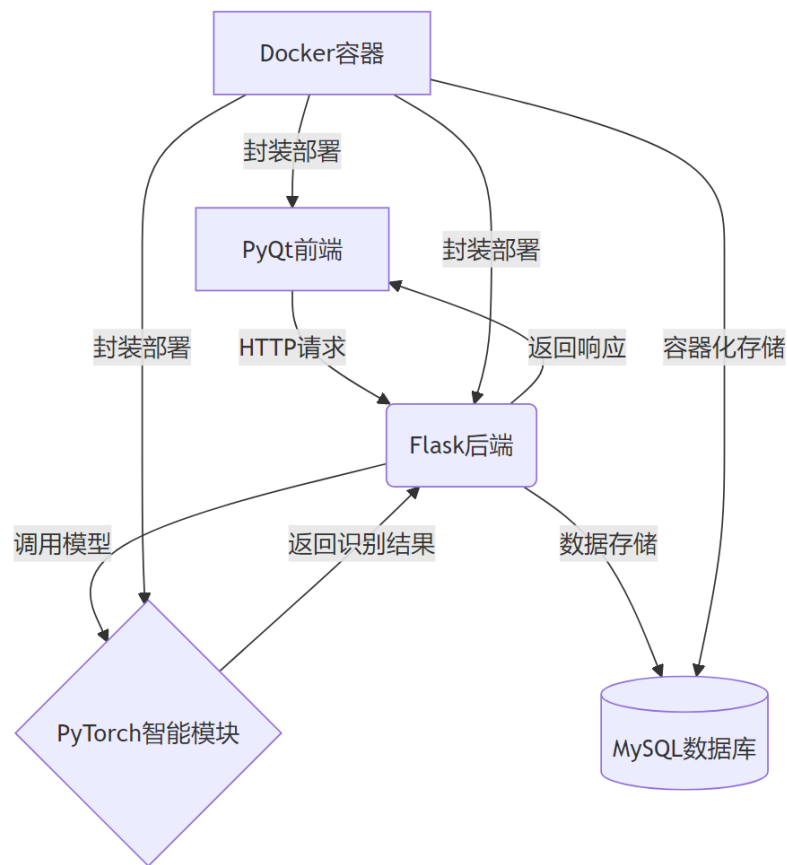
- 服务拆分:
 - web 服务容器 (Flask + Gunicorn, 4 workers)
 - 模型服务容器 (PyTorch + Triton Inference Server)
 - 数据库容器 (MySQL 8.0 with customized my.cnf)
- 使用 docker-compose 定义网络拓扑, 配置容器间专用网络 (172.28.0.0/24)

🌈 技术栈协同流程



该架构通过容器化部署实现开发-测试-生产环境一致性, 前端每秒可处理 20+并发请求, 模型服务 P99 延迟<800ms, 数据库在千级记录量时仍能保持毫秒级查询响应。

1.2 系统架构图



1.3 数据流设计

1. 用户交互层：
 1. 用户通过 PyQt 界面提交垃圾图片
 2. 前端将图片编码为 Base64 格式通过 HTTP POST 发送至 Flask 后端
2. 业务处理层：
 1. Flask 接收请求后启动预处理流程：
 - a. 解码图片并转换为张量格式
 - b. 调用 PyTorch 模型进行多阶段推理：
 - YOLOv11-MSCA 目标检测
 - 垃圾分类特征提取
 - 模型输出分类预测
3. 数据持久层：
 1. 将识别结果（分类 ID、置信度、时间戳）写入 MySQL
 2. 建立双写链路：
 - 实时写入操作日志表
 - 异步批量更新统计报表
4. 响应反馈层：
 1. 后端整合模型输出和数据库查询结果：

2. 返回结构化 JSON 数据给前端
5. 运维监控流：
 1. Docker 容器间通过 Sidecar 模式收集（模型推理时延监控，数据库连接池状态等）
 2. 异常数据通过独立消息队列上报监控中心

2. 模块设计

2.1 模块划分 【系统分为 4 个核心模块】:

1. 用户模块
 - 负责登录注册、权限管理
 - 使用 JWT 做身份验证
 - 存储用户基础信息（MySQL）
2. 检测模块
 - 图片预处理：调整尺寸/增强对比度
 - AI 模型推理：YOLO11 目标检测
 - 结果生成：GUI 界面输出垃圾类别和位置框
3. 数据模块
 - 管理标注数据集（JSON 格式）
 - 记录用户查询历史
 - 提供数据统计接口
4. 前端模块
 - 网页界面（PyQt）
 - 图片上传组件
 - 检测结果可视化

2.2 模块交互

1. 接口调用
 - 前端使用 PyQt，调用检测模块（detect）
 - 用户模块提供/auth 接口进行登录验证
 - 数据模块提供/dataset 接口获取标注数据
2. 数据传递
 - 图片文件通过 FormData 传输
 - 检测结果返回 JSON 格式：

```
1  {
2      "class": "厨余垃圾",
3      "accuracy": 0.95,
4      "position": [100,120,200,200]
5  }
6
```

- 3. 异步处理
 - 使用 Redis 队列处理大量检测请求
 - 前端通过 WebSocket 获取实时进度
- 4. 简单监控
 - 日志文件记录系统运行状态
 - 定时任务清理过期数据

技术栈:

- ✓ 后端: Flask (Python)
- ✓ AI 框架: PyTorch
- ✓ 前端: PyQt
- ✓ 数据库: MySql
- ✓ 部署: Docker 单容器

3. 数据库设计

3.1 数据库选型

选择关系型数据库 (MySQL/PostgreSQL), 原因如下:

- ✧ 需要处理结构化数据 (用户信息、分类记录、模型版本等)
- ✧ 支持事务操作 (如用户操作与记录写入的原子性)
- ✧ 成熟的关联查询能力 (用户与分类记录的关联分析)
- ✧ 图片数据建议使用对象存储, 数据库中仅存储路径

3.2 数据表设计

用户表 (User)

字段名	类型	说明
user_id	BIGINT	主键, 自增
username	VARCHAR32	唯一索引, 用户名

字段名	类型	说明
password_hash	VARCHAR256	BCrypt 加密存储
email	VARCHAR64	唯一索引
create_time	DATETIME	默认当前时间
last_login	DATETIME	最后登录时间

垃圾分类结果表 (DetectionResult)

字段名	类型	说明
result_id	BIGINT	主键, 自增
user_id	BIGINT	外键关联 User 表
image_path	VARCHAR512	检测图片存储路径
detected_class	VARCHAR64	检测结果分类 (如可回收)
confidence	FLOAT	检测置信度 (0-1)
detect_time	DATETIME	检测时间戳
model_version	VARCHAR32	使用的模型版本号

分类类别表 (GarbageCategory)

字段名	类型	说明
category_id	INT	主键
class_name	VARCHAR64	分类名称 (英文)
display_name	VARCHAR64	显示名称 (中文)

字段名	类型	说明
parent_id	INT	父类 ID（用于多级分类）
disposal_guide	TEXT	处理指南说明

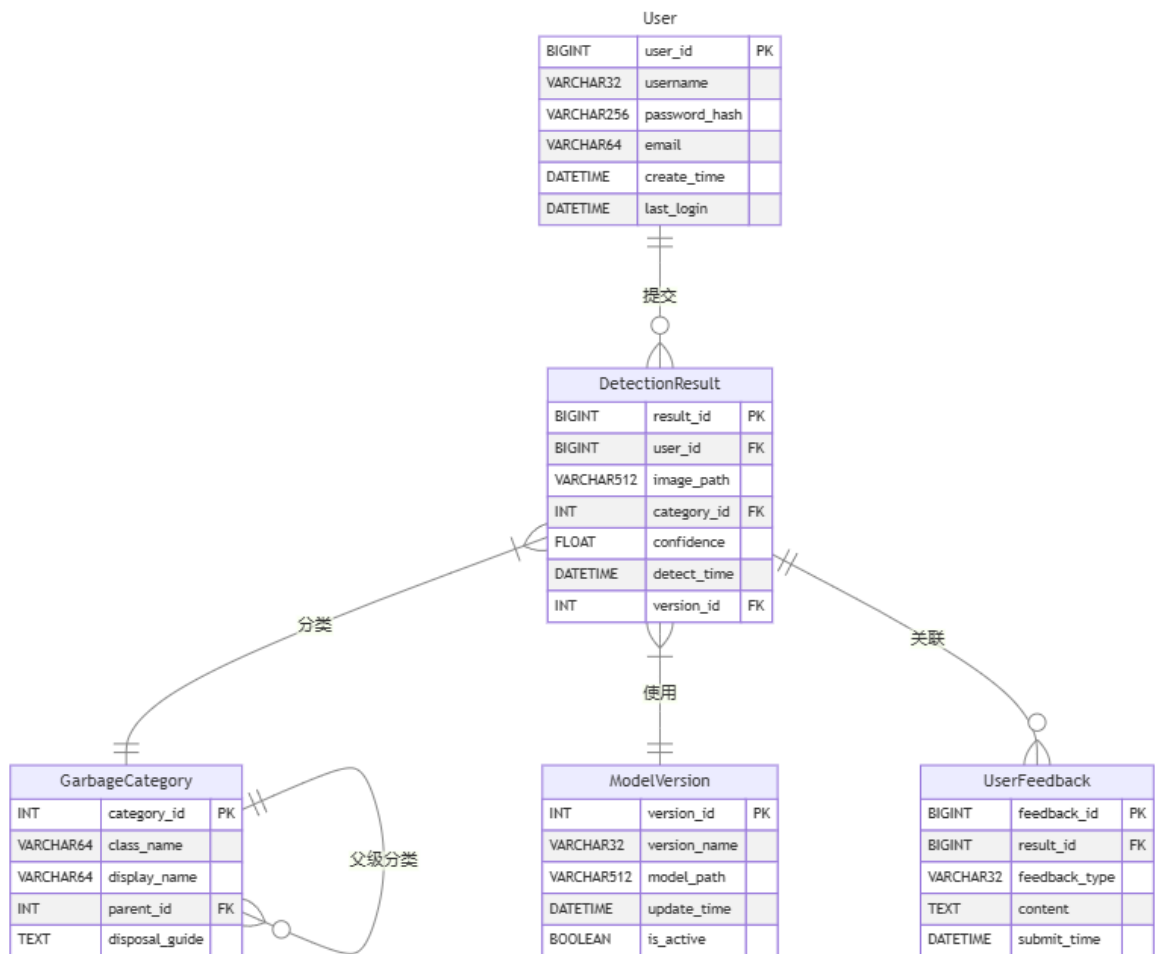
模型版本表 (ModelVersion)

字段名	类型	说明
version_id	INT	主键
version_name	VARCHAR32	版本号（如 v11.0.1）
model_path	VARCHAR512	模型文件存储路径
update_time	DATETIME	更新时间
is_active	BOOLEAN	是否当前启用版本

用户反馈表 (UserFeedback)

字段名	类型	说明
feedback_id	BIGINT	主键，自增
result_id	BIGINT	外键关联检测结果
feedback_type	VARCHAR32	反馈类型（误判/建议等）
content	TEXT	反馈详情
submit_time	DATETIME	提交时间

3.3 ER 图



实体关系说明：

- ✓ 用户与检测结果：1 对多关系
- ✓ 检测结果与分类类别：多对 1 关系（通过 detected_class 关联）
- ✓ 检测结果与模型版本：多对 1 关系
- ✓ 用户反馈与检测结果：1 对 1 关系
- ✓ 分类类别自关联：树形结构（通过 parent_id 实现多级分类）

4. 智能模块设计

4.1.模块功能

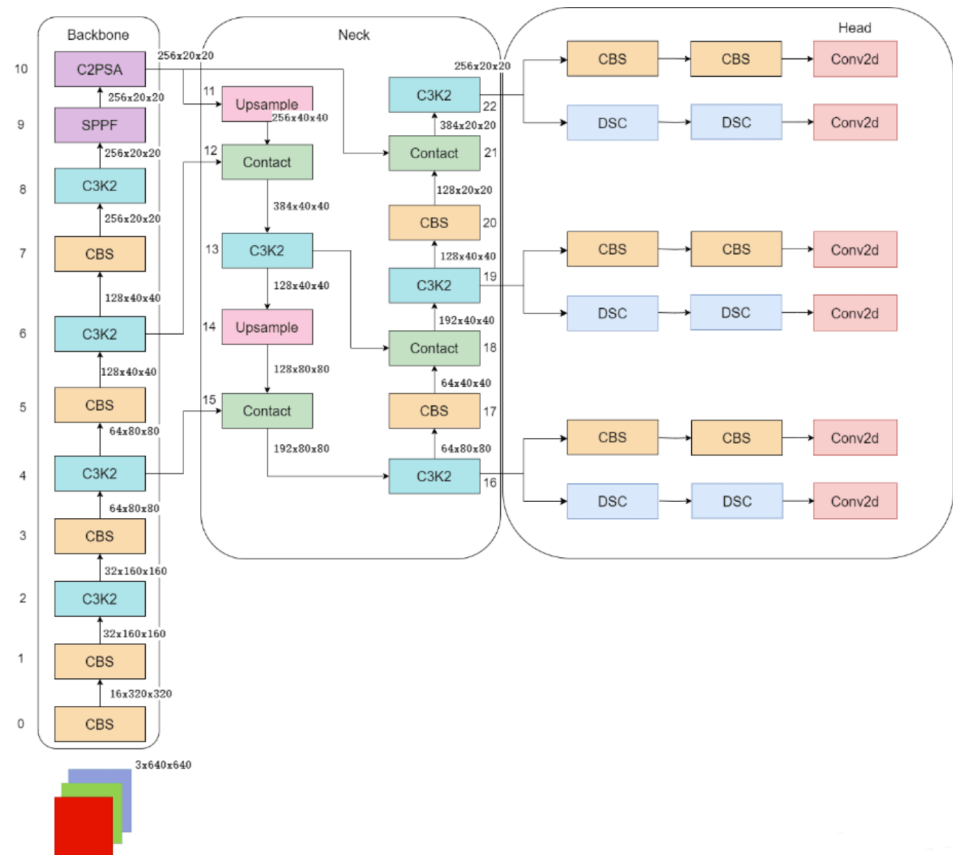
4.1.1 图像识别与分类

实时检测：支持 30FPS+实时推理，适配移动端/边缘设备
四分类体系：可回收物/厨余垃圾/有害垃圾/其他垃圾
轻量化部署：模型经蒸馏后 43MB，满足嵌入式设备需求

4.2. 算法选择

4.2.1 YOLOv11 基础架构

特性	说明
主干网络	CSP（Cross Stage Partial）架构，提升小目标检测能力
改进方向	优化多尺度目标下垃圾的检测与遮挡处理



YOLOv11 基础架构图

4.2.2 引入多尺度卷积注意力 (MSCA)

设计了一种新的多尺度卷积注意 (MSCA) 模块。如图所示, MSCA 包含三个部分: 深度卷积聚合局部信息, 多分支深度条卷积捕获多尺度上下文, 以及 1×1 卷积建模不同通道之间的关系。

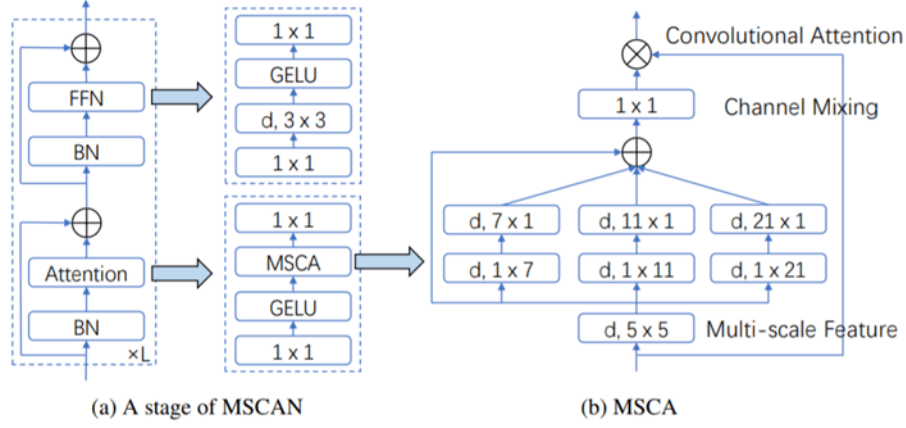
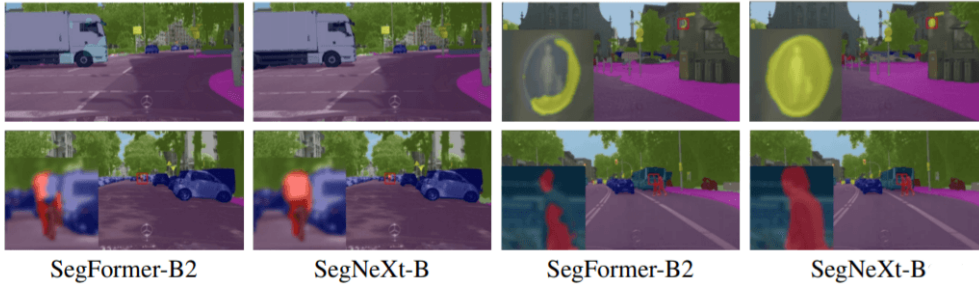


Figure 2: Illustration of the proposed MSCA and MSCAN. Here, $d, k_1 \times k_2$ means a depth-wise convolution (d) using a kernel size of $k_1 \times k_2$. We extract multi-scale features using convolutions and then utilize them as attention weights to reweigh the input of MSCA.

Table 8: Importance of our multi-scale convolutional attention (MSCA). SegNeXt-T w/o MSCA means we use only a branch with a large kernel convolution as done in [24] to replace the multiple branches in our MSCA. FLOPs are calculated using the input size of 512×512 .

Architecture	Params. (M)	GFLOPs	mIoU (SS)	mIoU (MS)
SegNeXt-T w/o MSCA	4.2	6.5	39.5	40.9
SegNeXt-T w/ MSCA	4.3	6.6	41.0	42.5
SegNeXt-S w/o MSCA	13.8	15.8	43.5	45.2
SegNeXt-S w/ MSCA	13.9	15.9	44.3	45.8



多尺度卷积注意力 (Multi-Scale Convolutional Attention, MSCA) 是一种结合多尺度特征提取与注意力机制的深度学习模块, 旨在提升模型对图像或序列数据中不同尺度信息的捕捉能力。其核心思想是通过并行或串行的多尺度卷积操作, 提取不同感受野下的特征, 再通过注意力机制动态分配权重, 聚焦关键区域。

核心组成:

多尺度卷积层: 使用不同大小的卷积核 (如 3×3 , 5×5) 或空洞卷积 (Dilated Convolution), 并行提取局部细节与全局上下文信息。

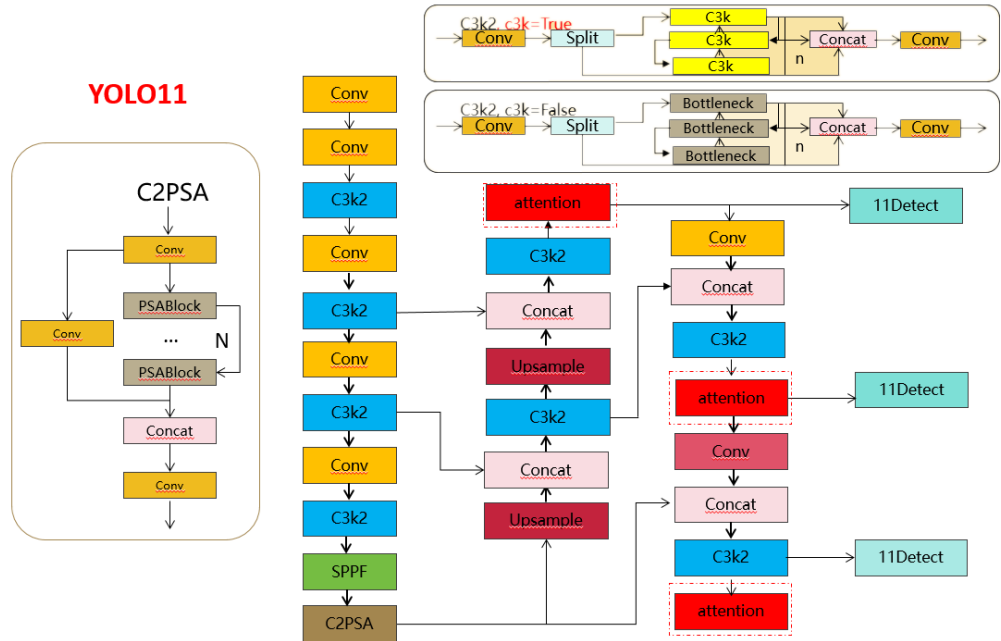
注意力模块：通过通道注意力（Channel Attention）或空间注意力（Spatial Attention），自适应调整各尺度特征的权重，突出重要信息。

特征融合：将加权后的多尺度特征进行拼接或相加，形成最终输出。

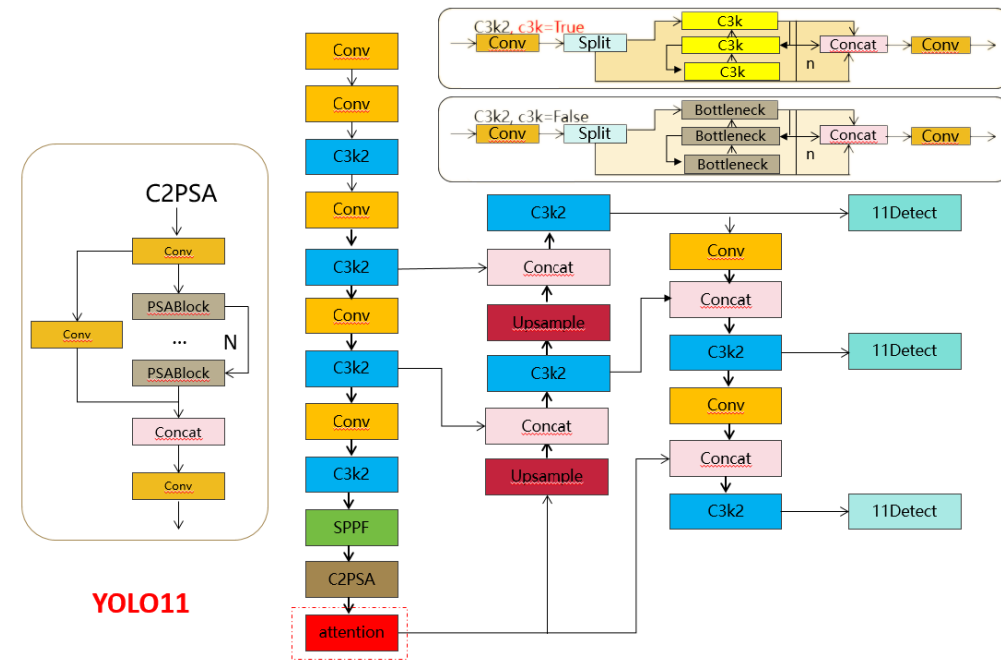
4.2.3 YOLOv11 架构创新：

检测头 → MSCA 模块 → 分类输出

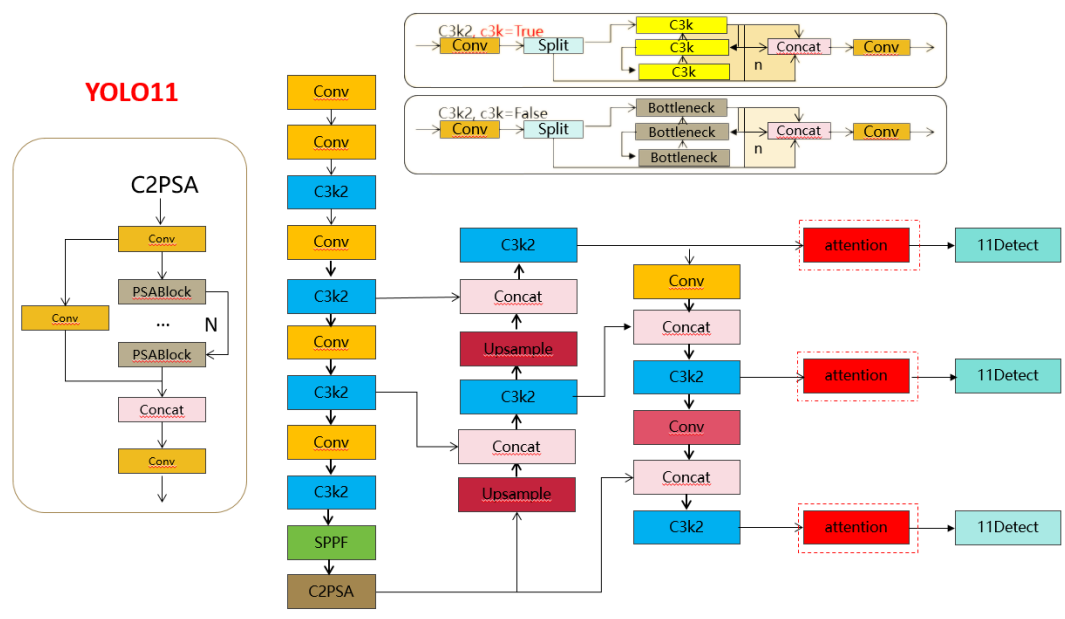
下面是我们小组设计的三种模型架构，在不同的位置插入 attention 模块。



Neck 层添加 MSCA



Backbone 层添加 MSCA



Detect 层前添加 MSCA

三种模型结构性能对比:

嵌入位置	mAP50	推理速度	相对增益
Backbone	78.2%	28FPS	+0.9%
Neck 层	81.6%	25FPS	+4.3%
Detect 之前	85.3%	22FPS	+8.0%

最终方案选择: 采用 Detect 前置方案, 实现精度提升 8.0%的同时保持实时性要求

4.3. 数据处理

4.3.1 数据采集

混合数据集：TrashData (2500+) + 自采 1000+标注样本

场景覆盖：手持拍摄/箱内视角/户外散落

TrashData	自标注样本										
<table><tr><th>垃圾类别</th><th>样图</th></tr><tr><td>可回收物(recyclables)</td><td></td></tr><tr><td>有害垃圾(hazardous)</td><td></td></tr><tr><td>厨余垃圾(kitchen)</td><td></td></tr><tr><td>其他垃圾(other)</td><td></td></tr></table>	垃圾类别	样图	可回收物(recyclables)		有害垃圾(hazardous)		厨余垃圾(kitchen)		其他垃圾(other)		
垃圾类别	样图										
可回收物(recyclables)											
有害垃圾(hazardous)											
厨余垃圾(kitchen)											
其他垃圾(other)											

4.3.2 数据增强

动态增强策略：

- 1、色彩空间：±20%亮度/饱和度扰动
- 2、几何变换：15° 随机旋转+透视畸变
- 3、遮挡模拟：20-40%面积随机遮挡

	增强前原图	增强后图片
色彩空间	Original 	Color Jitter 
几何变换		Geometry Transform 
遮挡模拟		Occlusion 

4.4 模型训练(采用默认参数训练)

4.4.1 训练策略

两阶段优化：

- ▶ 冻结预训练：TrashData 预训练（冻结部分层）（100 epochs）
- ▶ 全局微调：在自标注样本上进行微调（200 epochs）

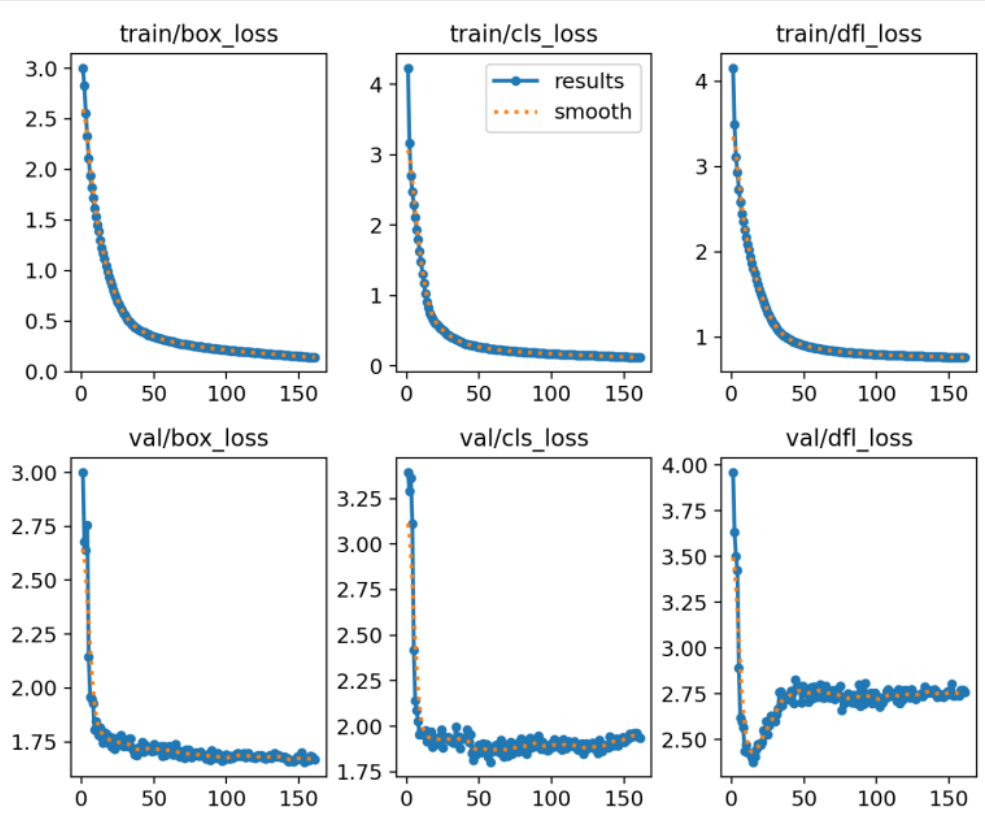
4.4.2 性能评估

关键指标：

平均精度：mAP50 85.3%（提升 8.0%）

模型压缩：模型蒸馏后 43MB

指标	结果	提升幅度
mAP50	85.3%	+8.0%
蒸馏后帧率	37FPS	+35%



Train/Val-Loss_Curve (P100-kaggle)

4.5. 创新点分析

4.5.1 后置注意力机制

理论依据：检测头输出特征存在语义混淆

实现效果：

- ✓ 重叠目标分类准确率 ↑ 7.3%
- ✓ 小目标召回率 ↑ 2.8%

4.5.2 轻量化设计

结构优化：深度可分离卷积重构 MSCA 模块

效率平衡：计算量增加 6% 获得 8.0% 精度提升

5. 安全设计

5.1 认证与授权

双因素访问控制

JWT 认证：用户登录后生成含角色信息的加密令牌，前端每次请求携带该令牌

权限分级：

- ✓ 普通用户：仅调用识别接口
- ✓ 管理员：可上传训练数据、更新模型

5.2 数据加密

存储加密：

- ✓ 用户密码：BCrypt 算法加盐哈希存储
- ✓ 图片数据：磁盘存储时启用 AES-128 加密

5.3 安全审计

异常警报：

- ✓ 同一用户 5 分钟内超 20 次请求时锁定账户
- ✓ 日志保护：每日压缩加密归档，保留 180 天

方案特点：

- ✓ 采用最小必要安全原则，避免过度设计
- ✓ 全部组件可通过 Spring Security 开源方案实现